Module 01: Deep Dive into Linear Algebra for Machine Learning

Cybernauts 2026 Datathon Campaign

May 2026

Abstract

Linear Algebra is not just about solving for x. In Machine Learning, it is the fundamental framework for representing data, performing efficient computations, and understanding the geometric structure of high-dimensional spaces. This note provides an intuitive and comprehensive guide to the concepts you need to win the datathon.

Contents

1 The Hierarchy of Data Structures in ML

In ML, we don't just see numbers; we see **tensors**. A tensor is simply a generalization of scalars, vectors, and matrices to any number of dimensions.

- **Scalars (0D):** A single magnitude. *Example:* The learning rate $\alpha = 0.001$.
- **Vectors (1D):** An ordered array of numbers. *Intuition:* A single data point. If you are predicting house prices, a vector $\mathbf{x} = [3, 1500, 2]^T$ represents one house with 3 bedrooms, 1500 sqft, and 2 baths.
- **Matrices (2D):** A grid of numbers. *Intuition:* A spreadsheet or a dataset. Rows are samples; columns are features.
- **Tensors (ND):** Higher-dimensional arrays. *Example:* A color image is a 3D tensor of shape $(Height, Width, Channels)$. A video is a 4D tensor adding the Time dimension.

2 Fundamental Operations: Alignment and Similarity

2.1 Matrix Transpose ($\mathbf{X}^T$)

Transposing a matrix flips it over its diagonal. Rows become columns. *Why it matters:* In ML, we often need to "align" dimensions. If you have a feature matrix $\mathbf{X}$ (Samples $\times$ Features) and you want to calculate the correlation between features, you often need $\mathbf{X}^T\mathbf{X}$.

2.2 The Vector Dot Product ($\mathbf{u} \cdot \mathbf{v}$)

The dot product is a single number: $\sum u_i v_i$.

The Geometric Intuition

The dot product measures **Similarity**. If two vectors point in the same direction, their dot product is large. If they are perpendicular (90°), it is zero. In ML, we use this to see how well a feature vector aligns with our model's weights.

Prediction Engine: $\hat{y} = \mathbf{w}^T\mathbf{x} + b$. This is just a dot product plus a bias.

2.3 Matrix Multiplication: Batch Processing

Multiplying a matrix $\mathbf{A}$ by a matrix $\mathbf{B}$ is like performing multiple dot products at once. *ML Perspective:* When we pass a whole batch of 32 images through a neural network layer, we aren't doing 32 separate calculations. We are doing one single **Matrix Multiplication**. This is why GPUs make ML so fast—they are "Matrix Multiplication Machines."

3 Systems of Equations and the Normal Equation

3.1 Linear Systems: $\mathbf{Xw} = \mathbf{y}$

Most of ML is trying to solve this equation. We have data $\mathbf{X}$ and targets $\mathbf{y}$, and we want to find the "importance" of each feature, $\mathbf{w}$. In most cases, we have more equations (samples) than unknowns (features). This is called an **overdetermined system**. There is usually no perfect solution that hits every point exactly, so we look for the "Best Fit."

3.2 The Closed-Form Normal Equation

To find the weights $\mathbf{w}$ that minimize the squared error, we use:

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{1}$$

When to apply: Use this for small-to-medium datasets where you want the **exact** best solution without using iterative Gradient Descent. It requires the matrix $(\mathbf{X}^T\mathbf{X})$ to be **Invertible**.

4 The Matrix Inverse and Identity Matrix

- **Identity Matrix (I):** The "number 1" of matrices. $\mathbf{AI} = \mathbf{A}$. It has 1s on the diagonal and 0s elsewhere.
- **Inverse ($\mathbf{A}^{-1}$):** The "reciprocal." $\mathbf{AA}^{-1} = \mathbf{I}$.

Warning: Not all matrices have an inverse. If a matrix has redundant columns (perfectly correlated features), it is "singular" and cannot be inverted. This is why we remove highly correlated features in data cleaning!

5 Eigen-everything: Variance and PCA

5.1 Eigenvalues and Eigenvectors

An eigenvector $\mathbf{v}$ is a direction that stays constant during a transformation. The eigenvalue λ is the factor by which it scales.

$$\mathbf{Av} = \lambda\mathbf{v} \tag{2}$$

5.2 The Covariance Matrix

The covariance matrix $\mathbf{\Sigma}$ summarizes how features vary together. If we take the eigenvectors of this matrix, we find the **Principal Components**.

5.3 Principal Component Analysis (PCA)

PCA is the king of dimensionality reduction.

1. It finds the eigenvector with the largest eigenvalue. This is the direction of **Maximum Variance**.
2. It projects the data onto these axes.

Intuition: If you have a 3D cloud of data that looks like a flat pancake, PCA will find the two directions that define the pancake and ignore the "thickness," reducing your 3D data to 2D without losing the main structure.

6 Vector Norms for Regularization

How do we stop a model from "cheating" by using massive weights to fit noise? We penalize the "size" of the weight vector $\mathbf{w}$.

- L_1 **Norm (Manhattan):** $\|\mathbf{w}\|_1 = \sum |w_i|$. *Effect:* It forces some weights to become exactly **zero**. Great for **Feature Selection**.
- L_2 **Norm (Euclidean):** $\|\mathbf{w}\|_2 = \sqrt{\sum w_i^2}$. *Effect:* It keeps weights **small and balanced**. This is the standard "Weight Decay" used in almost all deep learning models to prevent overfitting.

7 Practice Problems

P1: You have a dataset $\mathbf{X}$ where Column 1 is "Height in cm" and Column 2 is "Height in inches." Why will the Normal Equation fail?
Answer: The columns are linearly dependent (perfectly correlated). The matrix $\mathbf{X}^T\mathbf{X}$ will be singular and non-invertible.

P2: If you want to compress your data from 100 features to 10 features while keeping as much "information" as possible, which Linear Algebra tool do you use?
Answer: PCA, by selecting the 10 eigenvectors with the largest eigenvalues.